

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

João Pedro Moreno Alegre

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 06/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	4
4	Atividades realizadas	5
4.1	Atividade 1 — Agenda telefônica em Flask	5
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	7
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	9
4.4	Atividade 4 — Controle de acesso por papel — RBAC	13
4.5	Atividade 5 — Logs e auditoria	16
4.6	Atividade 6 — Upload e perfil de usuário	18
5	Atividades extras	21
5.1	Atividade E1 — Documentação Swagger/OpenAPI	21
5.2	Atividade E2 — CI/CD com GitHub Actions	23
5.3	Atividade E3 — Observabilidade — health checks e métricas	25
6	Considerações finais	28
7	Declaração de autoria	29

1 Introdução

Este relatório reúne as entregas que fiz no primeiro bimestre de Introdução à Computação em Nuvem (ISW055), disciplina da turma de Sistemas Inteligentes do período noturno da Fatec Pompeia, com o Prof. Allan Siriani, no segundo semestre de 2026. A disciplina ensina nuvem a partir de uma infraestrutura real, e não só pela teoria: cada aluno recebe um banco MariaDB que só ele acessa, um login no Portainer para publicar os próprios containers numa porta reservada e um subdomínio público. Todos usam o mesmo servidor ao mesmo tempo, isolados uns dos outros, que é o mesmo problema que um provedor de nuvem resolve quando atende vários clientes na mesma plataforma.

Depois de um exercício de nivelamento, uma agenda telefônica em Flask que guarda os contatos num arquivo JSON, o bimestre inteiro girou em torno de um único projeto: um catálogo dos filmes do Tom Hanks. A cada semana o catálogo ganhou uma necessidade que aparece em qualquer sistema de verdade. Primeiro, buscar dados numa API externa e guardar o que é de cada usuário sem misturar as contas. Depois, tirar o login de dentro da aplicação e colocá-lo num serviço separado, decidir as permissões pelo papel do usuário no backend, registrar quem fez o quê num serviço de auditoria e guardar as fotos de perfil num object storage. No fim do bimestre, o que começou como uma aplicação Flask virou cinco containers (catálogo, serviço de autenticação, serviço de logs, Redis e Garage) usando o MariaDB da disciplina, com só o catálogo exposto na internet, em joao-alegre-isw055.lapps.studio. Também fiz as três atividades extras propostas: documentação da API com Swagger, um pipeline de CI/CD no GitHub Actions e observabilidade com health checks e métricas.

O relatório está organizado assim: a seção 2 explica como as atividades foram feitas e de onde vêm as datas e horas usadas como prova; a seção 3 traz o quadro com a data planejada e a data realizada de cada entrega; a seção 4 tem uma ficha por atividade, com o link do commit, os prints e as dificuldades; a seção 5 documenta as atividades extras no mesmo formato; e a seção 6 fecha com o que aprendi no bimestre. Duas atividades foram entregues depois do prazo, a 1 e a 3, e aparecem assim no quadro e nas fichas, com o motivo.

2 Metodologia

Todos os serviços foram escritos em Python com Flask, e cada um tem o próprio Dockerfile. Para desenvolver, o `docker-compose.dev.yml` sobe o sistema inteiro localmente, junto com um MariaDB, um Redis e um Garage descartáveis, para que nenhum teste mexa nos dados de produção. Em produção, a `hanks-catalogue` roda no servidor da disciplina pelo Portainer, ligada ao repositório do GitHub e usando o banco MariaDB que recebi. Desde a atividade extra de CI/CD, as imagens são construídas pelo GitHub Actions e publicadas no GitHub Container Registry (GHCR), e o servidor só baixa a imagem do commit que vai para o ar.

Para testar, usei o navegador com o DevTools, o Postman (para chamar rotas protegidas sem passar pela interface), o Mailtrap (caixa de e-mail de teste para a recuperação de senha) e o pytest. Os dados dos filmes vêm da API do TMDb. Como fontes, usei os enunciados publicados no site da disciplina e a documentação oficial das ferramentas: Flask, Docker Compose, Portainer, Redis Streams, Garage, boto3, flasgger, GitHub Actions e prometheus-flask-exporter.

As atividades 2 a 6 e as extras estão num único repositório público, [joao-alegre07/Hanks-catalogue](https://github.com/joao-alegre07/Hanks-catalogue), porque são etapas do mesmo sistema, e cada etapa é um commit na branch `main`. A agenda telefônica (atividade 1) é um sistema diferente e ficou num repositório próprio, [joao-alegre07/agenda-telefonica](https://github.com/joao-alegre07/agenda-telefonica). O README dos dois menciona github.com/siriani. Para cada entrega do catálogo também abri uma issue no repositório

marcando o professor, com a descrição do que mudou e os prints, porque a menção no README sozinha não gerava notificação para ele.

A data planejada de cada atividade é a publicada na lista de atividades da disciplina. A data realizada é a data e a hora do commit que contém a entrega, tirada do histórico com `git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad"` e conferida na página do commit no GitHub. Todos os horários estão no horário de Brasília (GMT-3), o mesmo que o GitHub mostra na dica sobre a data do commit. Como o horário do commit vem do relógio do meu computador, conferi cada um com dois registros feitos pelo próprio GitHub: o horário em que o commit chegou ao GitHub, no [registro de atividade do repositório](#), e o horário em que a issue de entrega foi aberta. Esses horários aparecem embaixo da data realizada, em cada ficha. Nenhuma data de commit foi editada. Considerei “entregue” a atividade com commit até o dia planejado e “entregue com atraso” a que passou desse dia.

Cada ficha tem pelo menos dois prints. O primeiro é a página do commit no GitHub, em tela inteira, com o nome do repositório, o hash e a dica com a data e a hora exatas. Esses prints foram tirados durante a montagem deste relatório, em 05 e 06/10/2026: o relógio da barra de tarefas mostra quando o print foi tirado, e a data da entrega é a que aparece na dica. Os outros prints mostram o resultado e são, sempre que possível, os mesmos que anexe na issue de entrega, na época da entrega; alguns são recortes da parte da tela que importa. Quando um print foi tirado depois da entrega, a legenda diz isso.

3 Quadro de entregas

O quadro é montado a partir das fichas das seções 4 e 5. As atividades extras não têm prazo e aparecem no fim, com a letra E.

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	05/10/2026 20:55	entregue com atraso
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	20/08/2026 22:08	entregue
3	Desacoplando o login — microserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	01/09/2026 19:37	entregue com atraso
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	04/09/2026 16:39	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	23/09/2026 20:25	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	25/09/2026 20:48	entregue
E1	Documentação Swagger/OpenAPI	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.	sem prazo	01/10/2026 15:28	entregue
E2	CI/CD com GitHub Actions	Pipeline que testa e faz deploy a cada push.	sem prazo	05/10/2026 18:49	entregue
E3	Observabilidade — health checks e métricas	Endpoints de saúde e métricas, com o container reagindo à queda do Redis.	sem prazo	05/10/2026 20:10	entregue

Entregas com atraso:

- **Atividade 1:** faltei na aula de 07/08, em que a agenda foi feita em sala. Fiz a atividade sozinho em 05/10 e publiquei num repositório próprio, com a data real do commit.
- **Atividade 3:** passei a semana de 25/08 a 30/08 em São Paulo, a trabalho, e só consegui fazer a atividade na volta, em 01/09, quatro dias depois do prazo.

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	05/10/2026 20:55 commit 608425f no repositório agenda-telefonica (git log); chegou ao GitHub às 20:59
Situação	entregue com atraso faltei na aula de 07/08, em que a agenda foi feita em sala. Fiz a atividade sozinho em 05/10 e publiquei num repositório próprio, com a data real do commit.
Evidência	GitHub — commit e README no repositório agenda-telefonica; prints do sistema rodando localmente
Link	https://github.com/joao-alegre07/agenda-telefonica/commit/608425f

O que foi feito. A agenda é uma aplicação Flask única, sem API separada: as rotas renderizam os templates Jinja2 direto. São três telas: a lista de contatos em ordem alfabética, com busca pelo nome; o cadastro, com nome e telefone obrigatórios e e-mail opcional; e a consulta de um contato. Os contatos ficam no arquivo `contatos.json`, criado no primeiro cadastro e mantido fora do git pelo `.gitignore`. Ela roda localmente com `flask run`, como pedia o nivelamento, sem banco de dados e sem deploy.

No README deixei registrado por que o JSON resolve aqui e deixa de resolver no catálogo: a cada cadastro o arquivo inteiro é lido e regravado, dois acessos gravando ao mesmo tempo podem perder dados e não há como separar os contatos por usuário. É o que as atividades seguintes resolvem com o MariaDB.

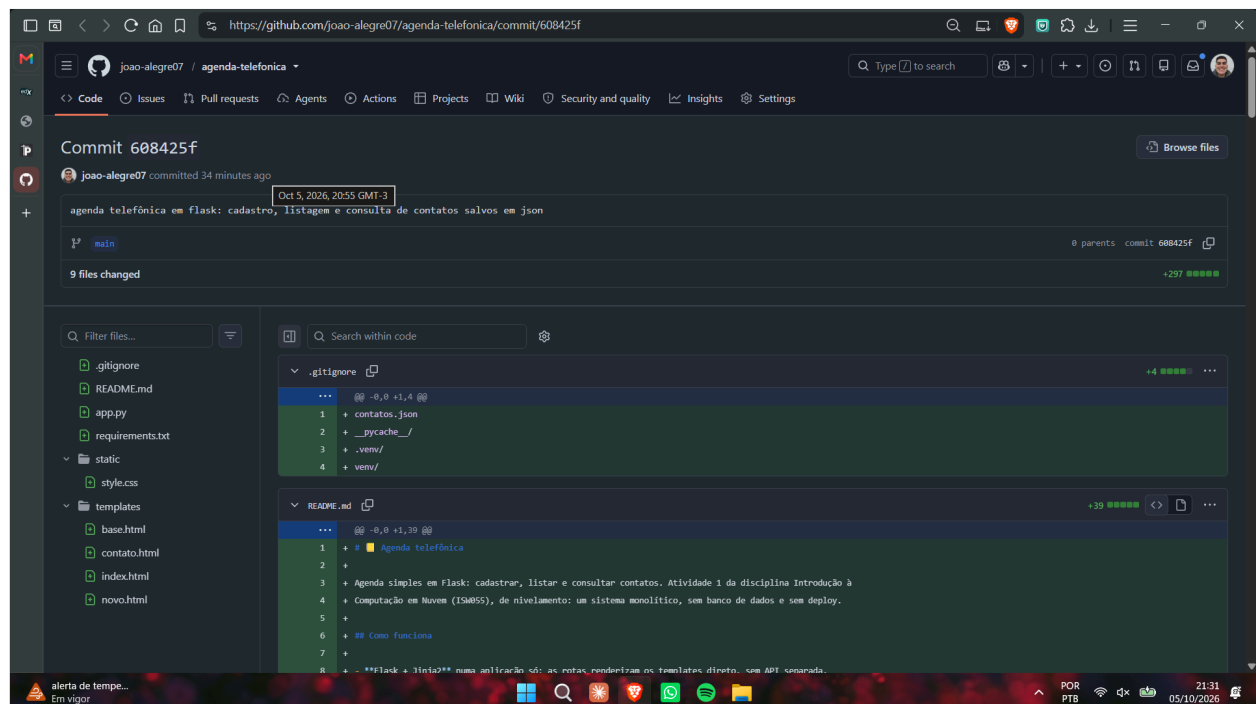


Figura 1: Atividade 1 — commit 608425f no repositório agenda-telefonica, com a dica mostrando 05/10/2026 20:55 (GMT-3).

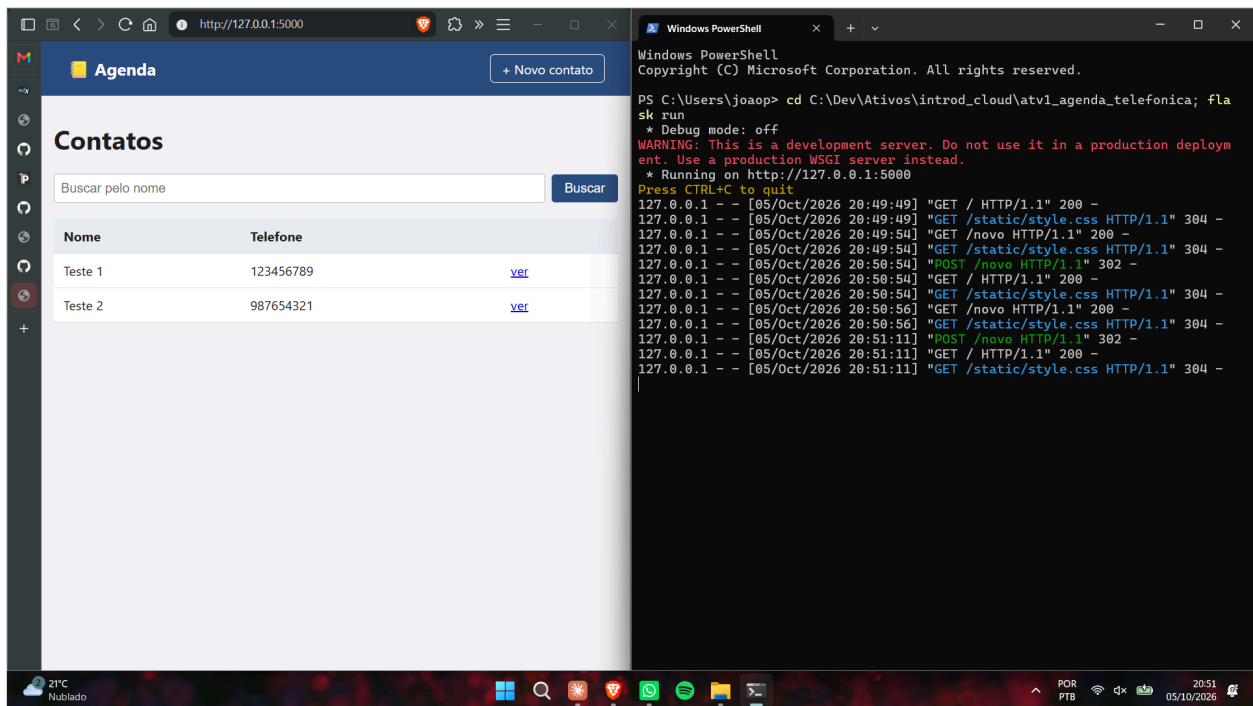


Figura 2: Atividade 1 — agenda rodando localmente (127.0.0.1:5000), com o terminal do flask run ao lado registrando as requisições. Print de 05/10/2026, 20:51.

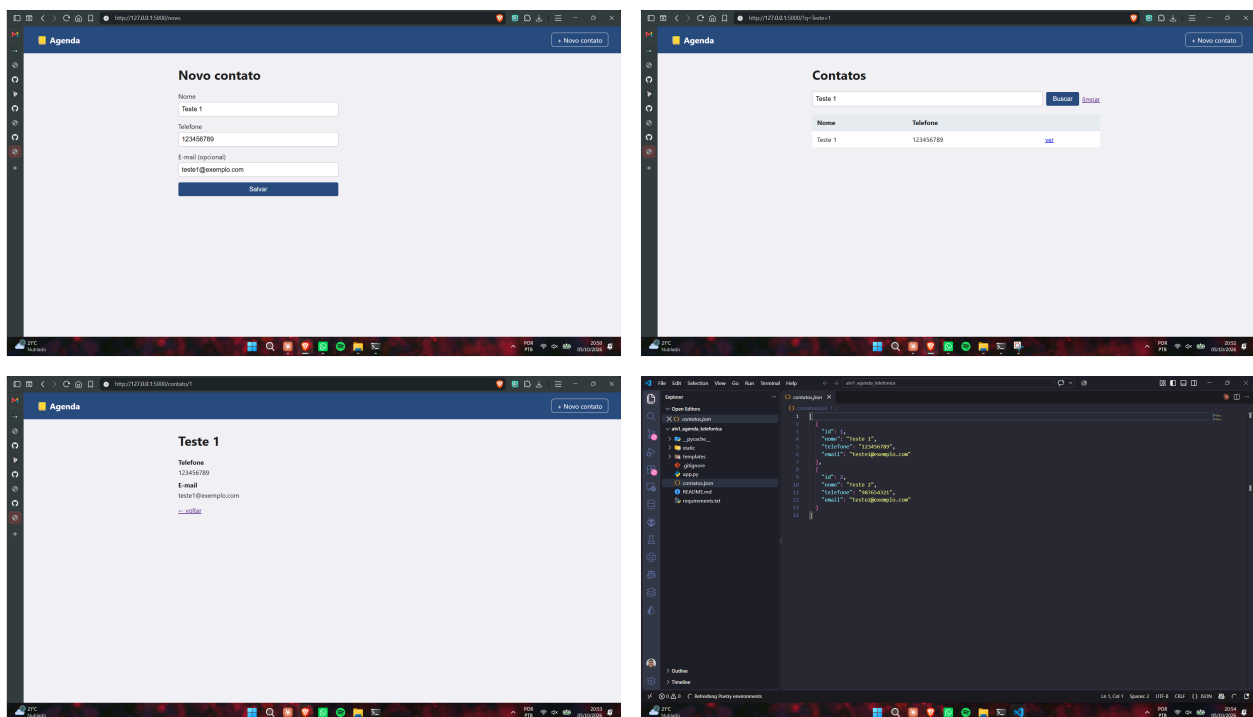


Figura 3: Atividade 1 — cadastro de um contato, busca pelo nome, consulta do contato e o `contatos.json` gravado pela aplicação. Prints de 05/10/2026, entre 20:50 e 20:54.

Dificuldades e como foram resolvidas. A dificuldade não foi técnica. Como faltei na aula em que a agenda foi feita, eu não tinha nenhum registro dela. Em vez de apresentar algo como se fosse da aula, refiz a atividade a partir do enunciado, mantive o escopo de nivelamento (sem banco, sem login e sem deploy) para não misturar com o que o catálogo trouxe depois e publiquei num repositório separado, com a data real do commit.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	20/08/2026 22:08 commit e0b6d19 (git log); chegou ao GitHub às 22:09; issue #1 aberta às 22:31
Situação	entregue
Evidência	GitHub — commit, README e issue #1; print do catálogo em produção
Link	https://github.com/joao-alegre07/Hanks-catalogue/commit/e0b6d19 https://github.com/joao-alegre07/Hanks-catalogue/issues/1

O que foi feito. O catálogo procura o Tom Hanks na API do TMDb (/search/person) e lista os filmes em que ele atua (/person/{id}/movie_credits), do mais recente para o mais antigo, 20 por página. A lista de filmes é sempre buscada ao vivo; no MariaDB ficam as contas e o que cada usuário faz, favoritos e comentários. O cadastro e o login são próprios, com a senha guardada como hash (werkzeug.security). O isolamento entre usuários está nas consultas: os favoritos são sempre filtrados pelo usuario_id da sessão, definida pelo servidor no login, e nunca por um id vindo da requisição.

Para publicar, escrevi o Dockerfile e dois arquivos compose: um de desenvolvimento, que sobe um MariaDB descartável, e um de produção, que o Portainer usa apontando para o banco da disciplina, na porta reservada 8212. A chave do TMDb e a senha do banco ficam só em variáveis de ambiente; o .env está no .gitignore e o .env.example vai sem valores. Na mesma noite, antes de marcar o professor, reorganizei o histórico em três commits (aplicação, Docker e README) para deixar as mensagens mais claras. Por isso o registro de atividade do GitHub mostra pushes a partir das 21:14 e a troca da branch main entre 22:09 e 22:15, e o commit da entrega ficou com o horário 22:08.

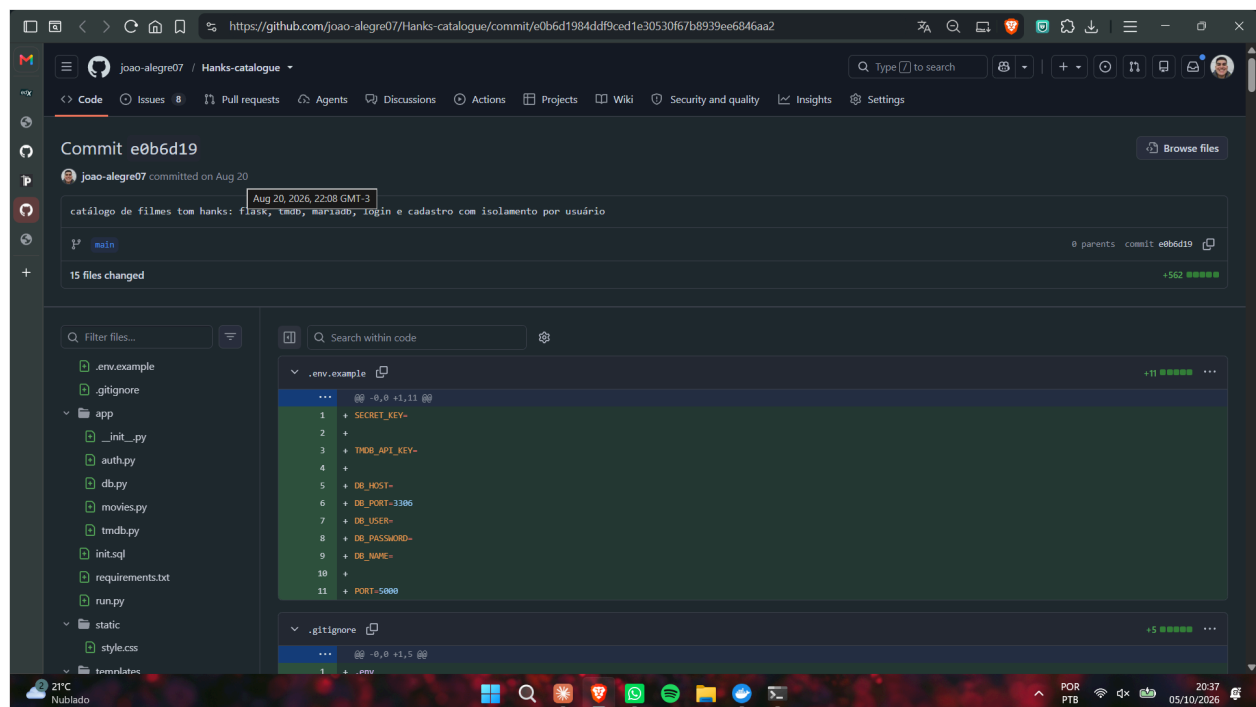


Figura 4: Atividade 2 — commit e0b6d19 no repositório Hanks-catalogue, com a dica mostrando 20/08/2026 22:08 (GMT-3).

O .env.example aberto mostra que nenhuma credencial foi versionada.

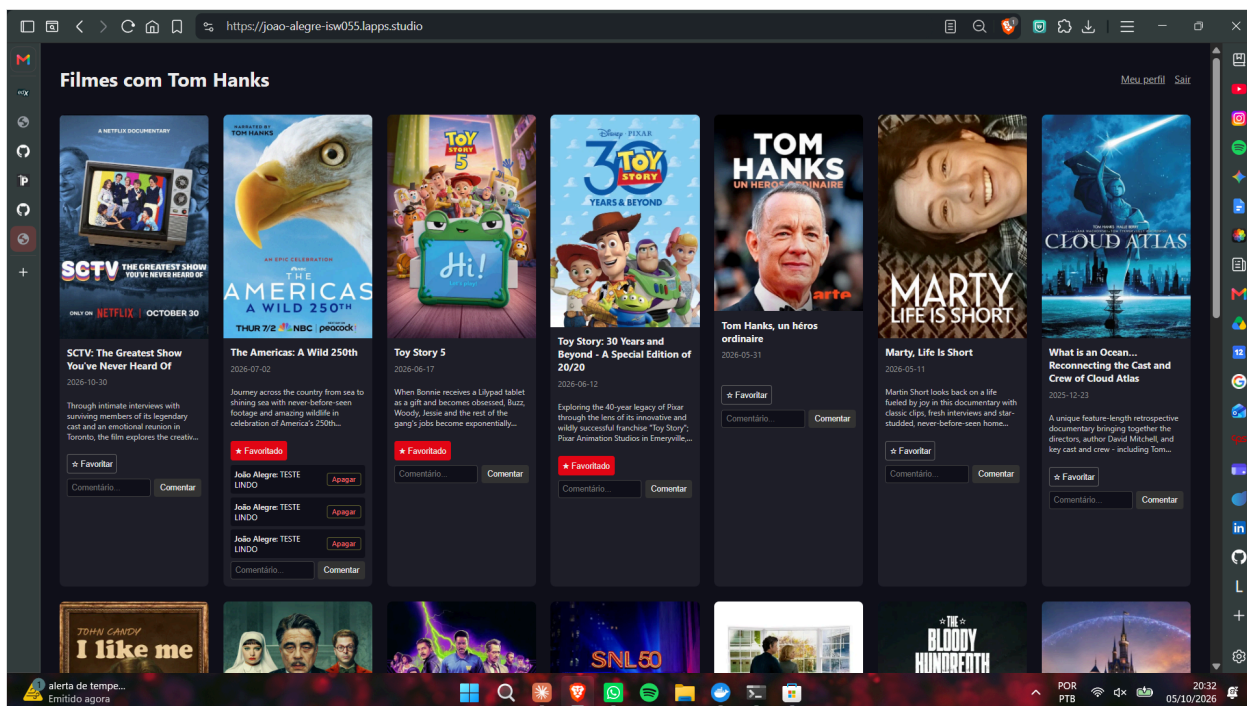


Figura 5: Atividade 2 — catálogo em produção, logado. Print de 05/10/2026: a tela já inclui itens das atividades seguintes, como o link “Meu perfil” e o botão de apagar comentário.

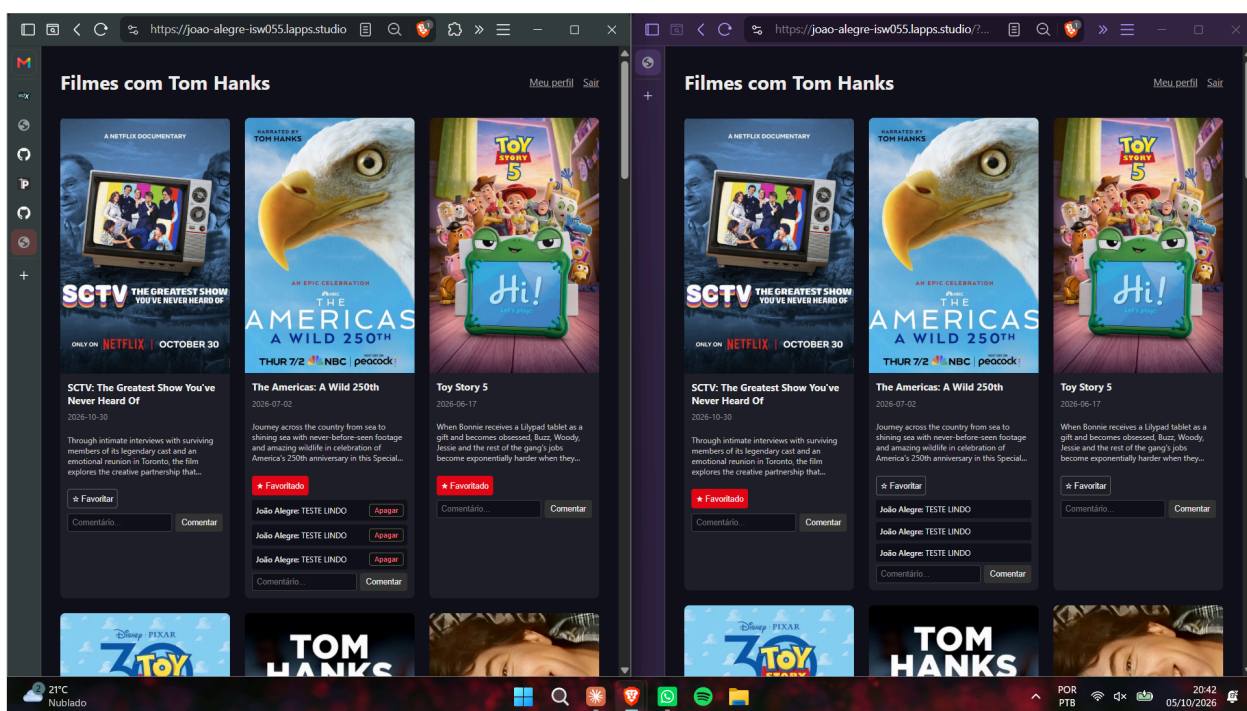


Figura 6: Atividade 2 — isolamento por usuário: duas contas ao mesmo tempo, a da direita numa janela privada. A da esquerda favoritou *The Americas* e *Toy Story 5*, a da direita favoritou *SCTV*, e nenhuma vê os favoritos da outra. Print de 05/10/2026.

Dificuldades e como foram resolvidas. O banco que recebi da disciplina ainda tinha tabelas de outro projeto, e o meu usuário só tem permissão no próprio schema: não dá para criar um segundo banco nem trocar a própria senha. Isso é o isolamento entre alunos funcionando, então removi as tabelas antigas e deixei só as do catálogo. No primeiro deploy, a sinopse aparecia cortada nos cards e todos os filmes carregavam numa página só, o que deixava as imagens lentas; ajustei o CSS e passei a paginar de 20 em 20. Também aprendi na prática que o Portainer publica o que está no repositório: um redeploy não mudou nada porque o commit ainda não tinha sido enviado ao GitHub.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	01/09/2026 19:37 commit 02c7f98 (git log); chegou ao GitHub às 19:38; ajuste do compose no commit f032f49, às 19:53; issue #2 aberta às 20:28
Situação	entregue com atraso passei a semana de 25/08 a 30/08 em São Paulo, a trabalho, e só consegui fazer a atividade na volta, em 01/09, quatro dias depois do prazo.
Evidência	GitHub — commit, docker-compose.yml e issue #2; prints do Portainer e do fluxo de login e de troca de senha
Link	https://github.com/joao-alegre07/Hanks-catalogue/commit/02c7f98 https://github.com/joao-alegre07/Hanks-catalogue/issues/2

O que foi feito. Tirei do catálogo tudo o que mexe com contas e criei o auth-service, outra aplicação Flask, com cadastro, login, papéis e recuperação de senha. O catálogo deixou de acessar a tabela de usuários: ele chama o auth-service por HTTP, pelo nome do serviço na rede interna do Docker (`http://auth:5001`). No compose, o auth-service não tem `ports:`, então não existe caminho de fora do servidor até ele; o único serviço com porta publicada continua sendo o catálogo, na 8212. A tabela de usuários ganhou o campo `role`, e todo cadastro nasce como `usuario`.

Na recuperação de senha, o auth-service gera um token aleatório, guarda na tabela `reset_tokens` com validade de 30 minutos e marca como usado depois da troca. O link chega por e-mail, com o SMTP configurado por variáveis de ambiente, e só funciona uma vez: link vencido ou já usado é recusado.

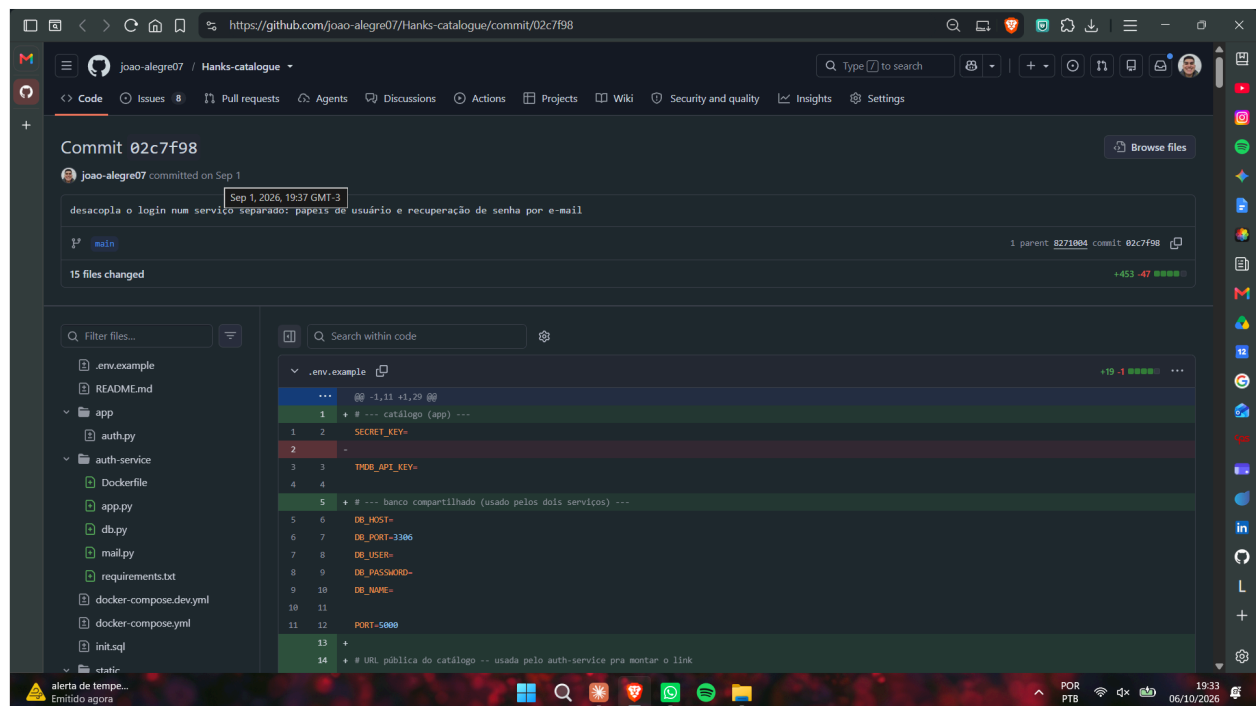


Figura 7: Atividade 3 — commit 02c7f98, com a dica mostrando 01/09/2026 19:37 (GMT-3).

```

10 services:
11   app:
12     build: .
13     ports:
14       - "8212:5000"
15     environment:
16       SECRET_KEY: ${SECRET_KEY}
17       TMDB_API_KEY: ${TMDB_API_KEY}
18       DB_HOST: ${DB_HOST}
19       DB_PORT: ${DB_PORT}
20       DB_USER: ${DB_USER}
21       DB_PASSWORD: ${DB_PASSWORD}
22       DB_NAME: ${DB_NAME}
23       AUTH_SERVICE_URL: http://auth:5001
24       PUBLIC_APP_URL: ${PUBLIC_APP_URL}
25     depends_on:
26       - auth
27     restart: unless-stopped
28
29   # Serviço de autenticação: login, cadastro, papéis (role) e esqueci-senha.
30   # Sem "ports:" de propósito -- só acessível pela rede do projeto, nunca
31   # publicado pro host. O catálogo é o único ponto de entrada público.
32   auth:
33     build: ./auth-service
34     environment:
35       DB_HOST: ${DB_HOST}
36       DB_PORT: ${DB_PORT}
37       DB_USER: ${DB_USER}
38       DB_PASSWORD: ${DB_PASSWORD}
39       DB_NAME: ${DB_NAME}
40       SMTP_HOST: ${SMTP_HOST}
41       SMTP_PORT: ${SMTP_PORT}
42       SMTP_USER: ${SMTP_USER}
43       SMTP_PASSWORD: ${SMTP_PASSWORD}
44       MAIL_FROM: ${MAIL_FROM}
45     restart: unless-stopped

```

Figura 8: Atividade 3 — docker-compose.yml com os dois serviços: o catálogo publica a porta 8212 e o auth não tem ports:, só é alcançado pela rede interna.

Name ↓↑	State ↓↑ Filter ▾	Quick Actions	Stack ↓↑	Image ↓↑	Created ↓↑	IP Address ↓↑	Published Ports ↓↑	Ownership ↓↑
hanks-catalogue-app-1	running	🔄 🏠 🔍 🗑️	hanks-catalogue	hanks-catalogue-app	2026-09-01 19:55:47	192.168.32.3	🔗 8212:5000	🔒 private
hanks-catalogue-auth-1	running	🔄 🏠 🔍 🗑️	hanks-catalogue	hanks-catalogue-auth	2026-09-01 19:55:37	192.168.32.2	-	🔒 private

Figura 9: Atividade 3 — containers da stack no Portainer, criados em 01/09/2026 às 19:55: o app com a porta 8212 publicada e o auth sem nenhuma porta (coluna *Published Ports*).

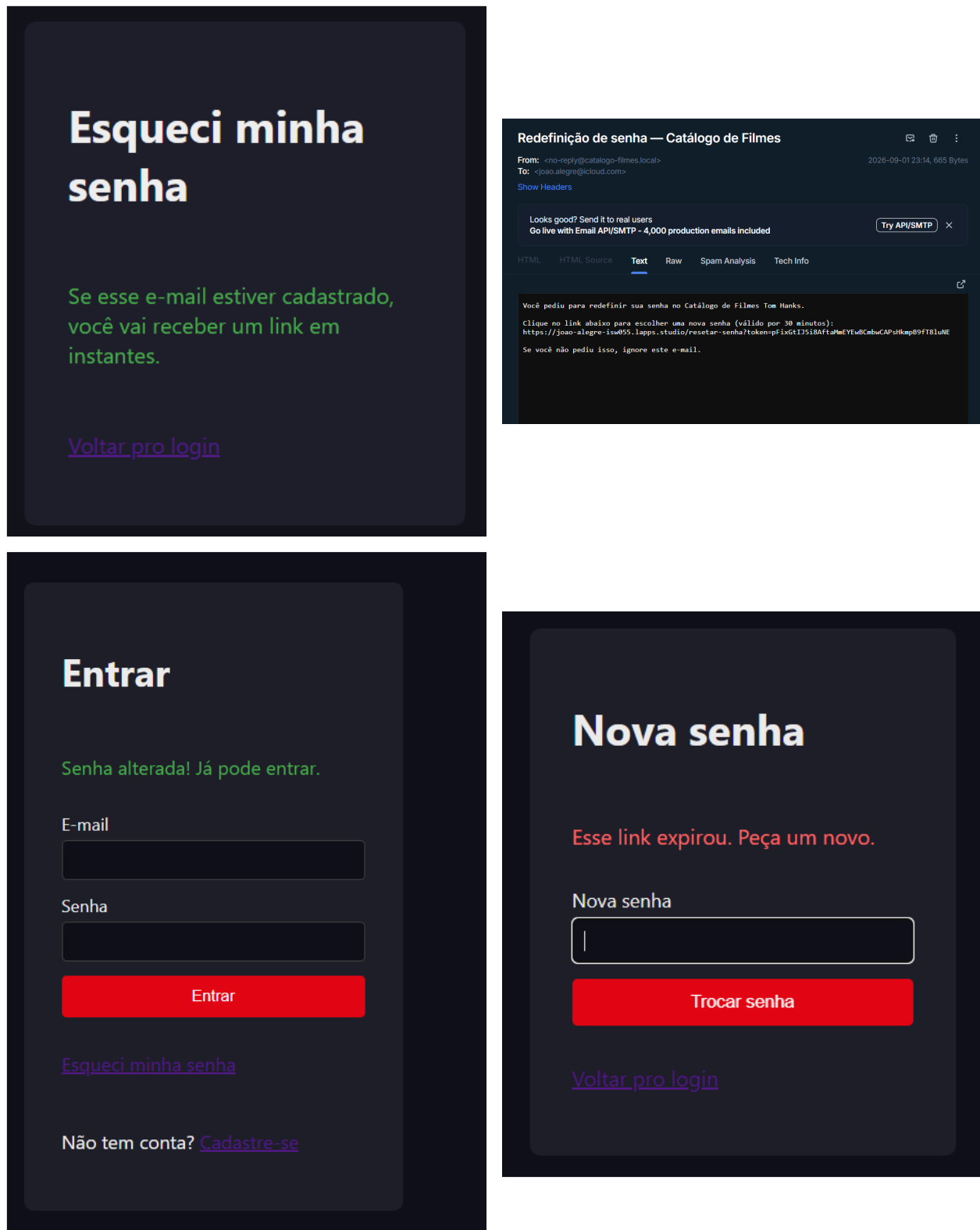


Figura 10: Atividade 3 — recuperação de senha em produção: pedido do link, e-mail recebido no Mailtrap em 01/09/2026, login depois da senha trocada e tentativa com link expirado recusada.

Dificuldades e como foram resolvidas. O primeiro deploy da stack falhou e eu só percebi depois, porque o Portainer mostra o erro numa notificação no canto da tela. O compose criava uma rede própria para os serviços, e o servidor compartilhado não tinha sub-rede livre para criar outra. Tirei a rede customizada e passei a usar a rede padrão do projeto, que já isola os containers da stack (commit f032f49, às 19:53). Também precisei confirmar que o auth estava mesmo sem porta pública, já que o Portainer lista os

containers de todos os alunos; conferi pela coluna de portas dos containers da minha stack. Por fim, o e-mail de teste não chegava na minha caixa pessoal: o Mailtrap guarda as mensagens numa caixa de teste própria, e foi lá que conferi o link.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	04/09/2026 16:39 commit be82747 (git log); chegou ao GitHub às 16:39; issue #3 aberta às 17:19
Situação	entregue
Evidência	GitHub — commit, README e issue #3; prints do 403 e da ação de admin no Postman
Link	https://github.com/joao-alegre07/Hanks-catalogue/commit/be82747 https://github.com/joao-alegre07/Hanks-catalogue/issues/3

O que foi feito. O role criado na atividade 3 passou a decidir o que cada usuário pode fazer. A regra escolhida foi a moderação de comentários: o usuário comum apaga só os próprios comentários, e o admin apaga o de qualquer pessoa. Para a moderação fazer sentido, os comentários passaram a ser visíveis para todos os usuários logados (até então cada um via só os seus); os favoritos continuaram privados.

A checagem fica no backend. Na rota POST /comentarios/<id>/deletar, o catálogo pergunta ao auth-service (GET /usuarios/<id>) qual é o papel atual de quem está pedindo e responde 403 se a pessoa não for a autora nem admin, independente do que a interface mostra. No README documentei a tabela de permissões por papel e a comparação entre os padrões de autorização: o meu é o Padrão A, com a decisão centralizada no auth-service, que passa a valer na hora em que o papel de alguém muda. Também expliquei o que mudaria com o Padrão B (papel dentro de um JWT assinado no login): sem a chamada extra a cada pedido, mas com a mudança de papel só valendo quando o token expira.

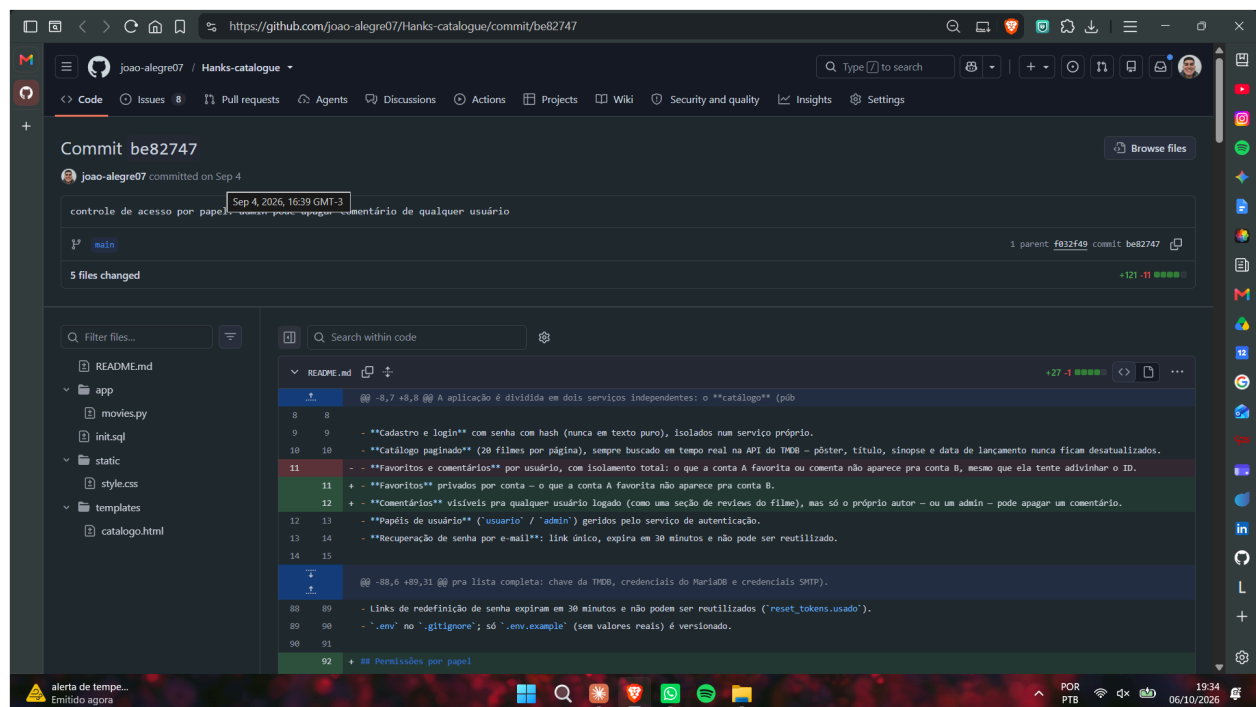


Figura 11: Atividade 4 — commit be82747, com a dica mostrando 04/09/2026 16:39 (GMT-3).

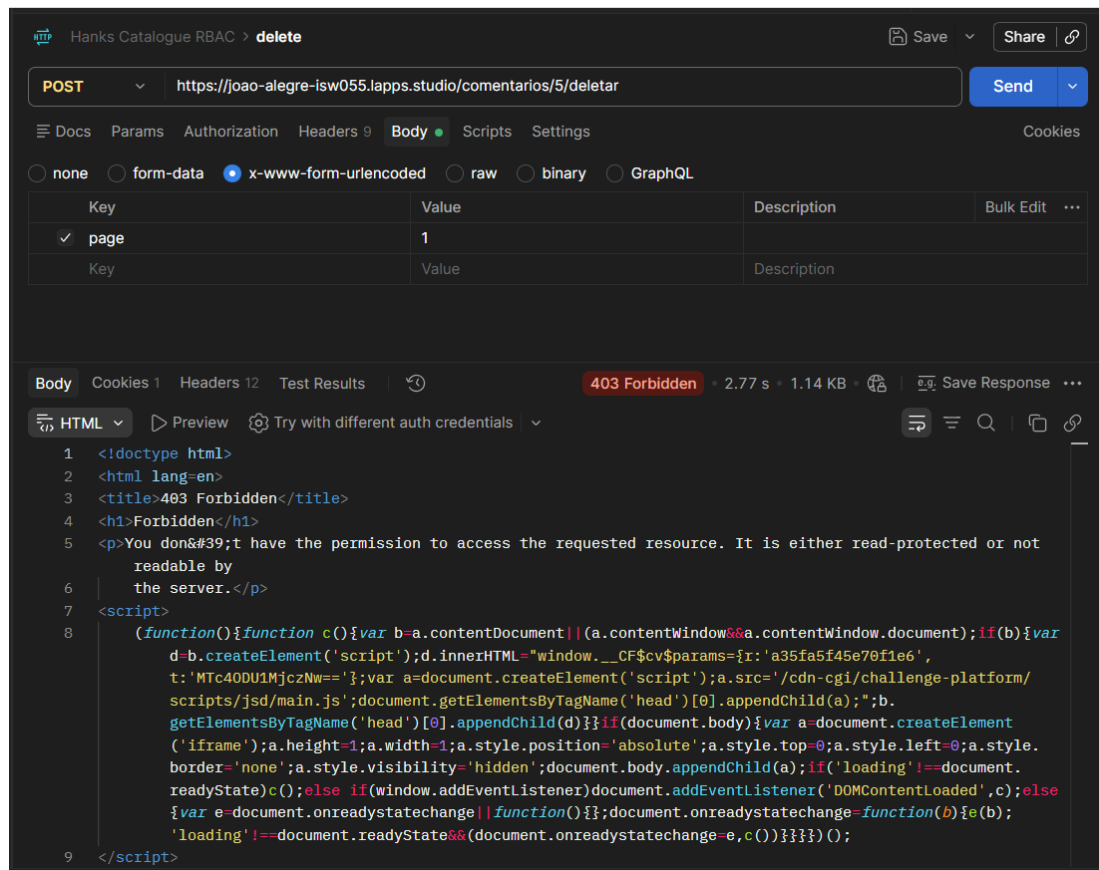


Figura 12: Atividade 4 — usuário comum chamando POST /comentarios/5/deletar direto pelo Postman, sem passar pela interface: 403 Forbidden.

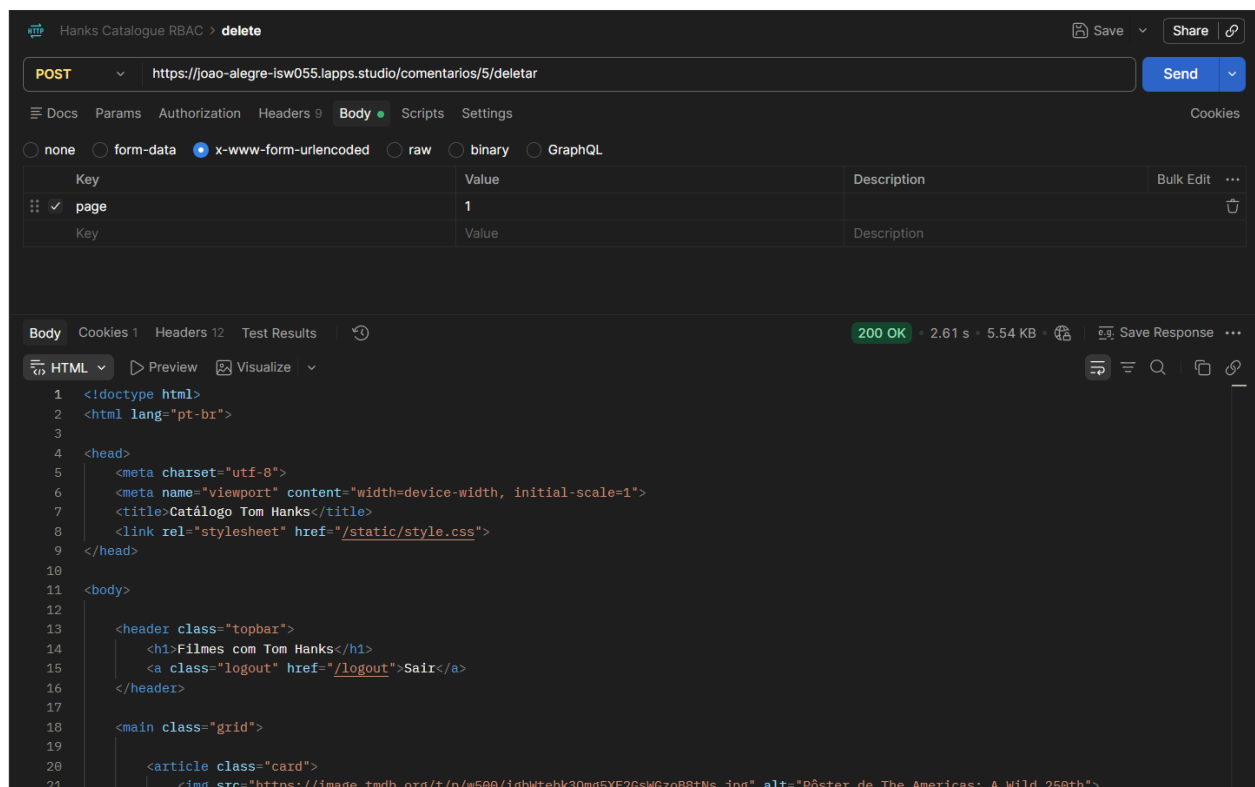


Figura 13: Atividade 4 — o mesmo pedido com a sessão de um admin: 200 OK. O comentário é apagado e o corpo da resposta é o catálogo, para onde a rota redireciona.

Dificuldades e como foram resolvidas. Eu queria provar o 403 sem depender da interface, então usei o Postman pela primeira vez, enviando o cookie de sessão de cada conta. No pedido do admin, a resposta veio com cerca de 600 linhas, porque depois de apagar a rota redireciona para o catálogo e o Postman segue o redirecionamento; o que serve de prova é o 200 contra o 403 do usuário comum. A outra questão foi mudar a visibilidade dos comentários sem desfazer o isolamento da atividade 2: os favoritos continuaram privados, e o botão “Apagar” só aparece para o autor e para o admin, com o backend conferindo de novo a cada pedido.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	23/09/2026 20:25 commit 7e91603 (git log); chegou ao GitHub às 20:26; issue #4 aberta às 21:41
Situação	entregue
Evidência	GitHub — commit, README e issue #4; print da consulta dos logs pelo admin
Link	https://github.com/joao-alegre07/Hanks-catalogue/commit/7e91603 https://github.com/joao-alegre07/Hanks-catalogue/issues/4

O que foi feito. Criei o log-service, um terceiro serviço Flask que recebe os eventos de auditoria por HTTP e grava num Redis Stream (XADD). O catálogo e o auth-service mandam um evento a cada ação relevante: login e login com falha, logout, favoritar e desfavoritar, comentar, apagar comentário, moderação feita por admin e toda resposta 403. Cada evento guarda o usuário, a ação, o horário, o IP de origem, o serviço que enviou e um campo de detalhes. O log-service e o Redis não têm porta publicada, e só o log-service escreve no Redis, que roda com appendonly yes e volume próprio para os eventos sobreviverem a um restart.

O acesso_negado é registrado num handler de erro 403 do Flask, e não rota por rota, então qualquer rota nova que devolver 403 já entra no log. A consulta fica em /admin/logs e usa o mesmo controle de acesso da atividade 4: quem não é admin recebe 403, e essa tentativa também é registrada.

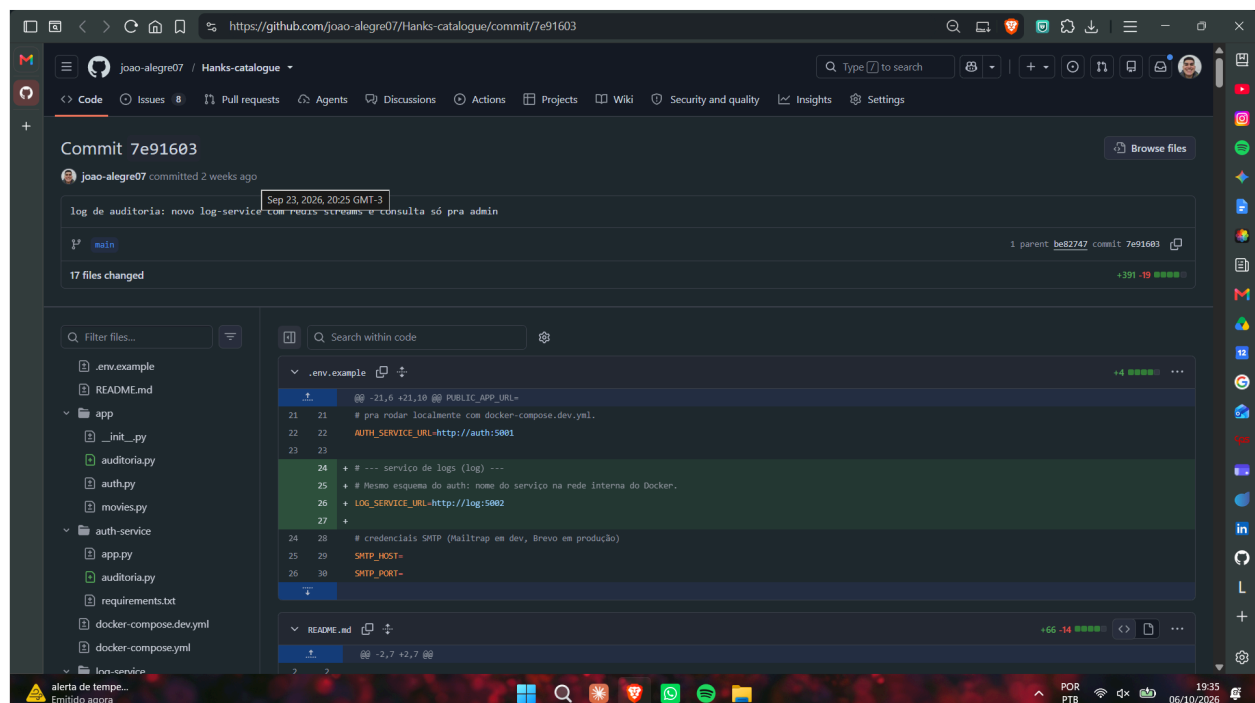


Figura 14: Atividade 5 — commit 7e91603, com a dica mostrando 23/09/2026 20:25 (GMT-3).

```

52 # Serviço de auditoria: recebe os eventos do catálogo e do auth e grava
53 # num Redis Stream. Também sem porta publicada -- a consulta dos logs
54 # passa pelo catálogo, na rota /admin/logs, que só admin acessa.
55 log:
56   build: ./log-service
57   environment:
58     REDIS_URL: redis://redis:6379/0
59   depends_on:
60     - redis
61   restart: unless-stopped
62
63 # Só o log-service fala com o Redis. O appendonly faz o Redis gravar cada
64 # escrita em disco, então os eventos sobrevivem a um restart do container.
65 redis:
66   image: redis:7-alpine
67   command: ["redis-server", "--appendonly", "yes"]
68   volumes:
69     - redis_data:/data
70   restart: unless-stopped
71
72 volumes:
73   redis_data:
74

```

Figura 15: Atividade 5 — trecho do docker-compose.yml com o log-service e o Redis, os dois sem porta publicada e o Redis com appendonly.

Log de auditoria

[Catálogo](#) [Sair](#)

Últimos 50 eventos, do mais recente pro mais antigo.

Horário (UTC)	Usuário	Ação	Serviço	IP	Detalhes
2026-09-23T23:50:05+00:00	14	login	auth	189.90.97.245	
2026-09-23T23:50:01+00:00	14	login	auth	189.90.97.245	
2026-09-23T23:49:38+00:00	15	logout	catalogo	189.90.97.245	
2026-09-23T23:49:33+00:00	15	acesso_negado	catalogo	189.90.97.245	POST /comentarios/6/deletar
2026-09-23T23:49:16+00:00	15	acesso_negado	catalogo	189.90.97.245	GET /admin/logs
2026-09-23T23:49:08+00:00	15	comentar	catalogo	189.90.97.245	comentário 7 no filme 1725605
2026-09-23T23:48:58+00:00	15	favoritar	catalogo	189.90.97.245	filme 1725605 (The Americas: A Wild 250th)
2026-09-23T23:48:24+00:00	15	login	auth	189.90.97.245	
2026-09-23T23:47:57+00:00	—	login_falhou	auth	189.90.97.245	joapedromorenoalegre@gmail.com
2026-09-23T23:45:58+00:00	3	login_falhou	auth	189.90.97.245	joao.alegre@icloud.com

Figura 16: Atividade 5 — /admin/logs aberto pelo admin em 23/09/2026. De baixo para cima: dois logins com falha, o login de uma conta comum, favoritar, comentar, as duas tentativas negadas (abrir /admin/logs e apagar o comentário de outro usuário), o logout e o login do admin. Horários em UTC (23:48 UTC são 20:48 em Brasília).

Dificuldades e como foram resolvidas. Para testar a consulta eu precisava de uma conta admin, e o sistema não tem como alguém virar admin pela interface, porque todo cadastro nasce como usuario, de propósito. Promovi a minha conta direto no banco, com um UPDATE no campo role. Outra decisão foi o horário dos eventos: como eles vêm de dois serviços, o horário é definido pelo log-service quando recebe o evento, sempre em UTC, para todos ficarem na mesma linha do tempo; por isso a tela mostra UTC. Também não quis que o log virasse um ponto único de falha: o envio do evento tem timeout curto e, se o log-service estiver fora do ar, a ação do usuário continua funcionando.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	25/09/2026 20:48 commit 591daf1 (git log); chegou ao GitHub às 20:49; prints e issue #5 em 30/09, às 20:05
Situação	entregue
Evidência	GitHub — commit, README e issue #5; prints do perfil com foto e do 403 ao editar outro perfil
Link	https://github.com/joao-alegre07/Hanks-catalogue/commit/591daf1 https://github.com/joao-alegre07/Hanks-catalogue/issues/5

O que foi feito. Cada usuário ganhou uma página de perfil (/perfil/<id>) com nome, foto, bio e os filmes que favoritou, e o nome de quem comentou no catálogo leva para o perfil da pessoa. No lugar do MinIO usei o Garage, um object storage compatível com S3, leve e pensado para rodar num nó só. Ele é mais um serviço do compose, sem porta publicada, e no primeiro boot cria o bucket e a chave de acesso a partir das variáveis de ambiente. A foto vai para o Garage e o MariaDB guarda só a bio e a chave do objeto (algo como perfis/3/<uuid>.png); cada upload gera uma chave nova, e o objeto antigo é apagado depois que o banco já aponta para o novo.

O upload aceita JPG, PNG ou WebP de até 2 MB, e o tipo é conferido pelos primeiros bytes do arquivo, não pela extensão nem pelo Content-Type que o navegador manda. Para exibir a foto escolhi bucket privado com URL pré-assinada, válida por 10 minutos; como o Garage não tem porta pública, a URL passa pela rota /fotos do catálogo, que só repassa o pedido, e quem valida a assinatura e a validade é o próprio Garage. Qualquer usuário logado vê qualquer perfil, mas só edita o próprio: a rota de edição compara o id da URL com o usuário da sessão e responde 403 se forem diferentes, até para admin.

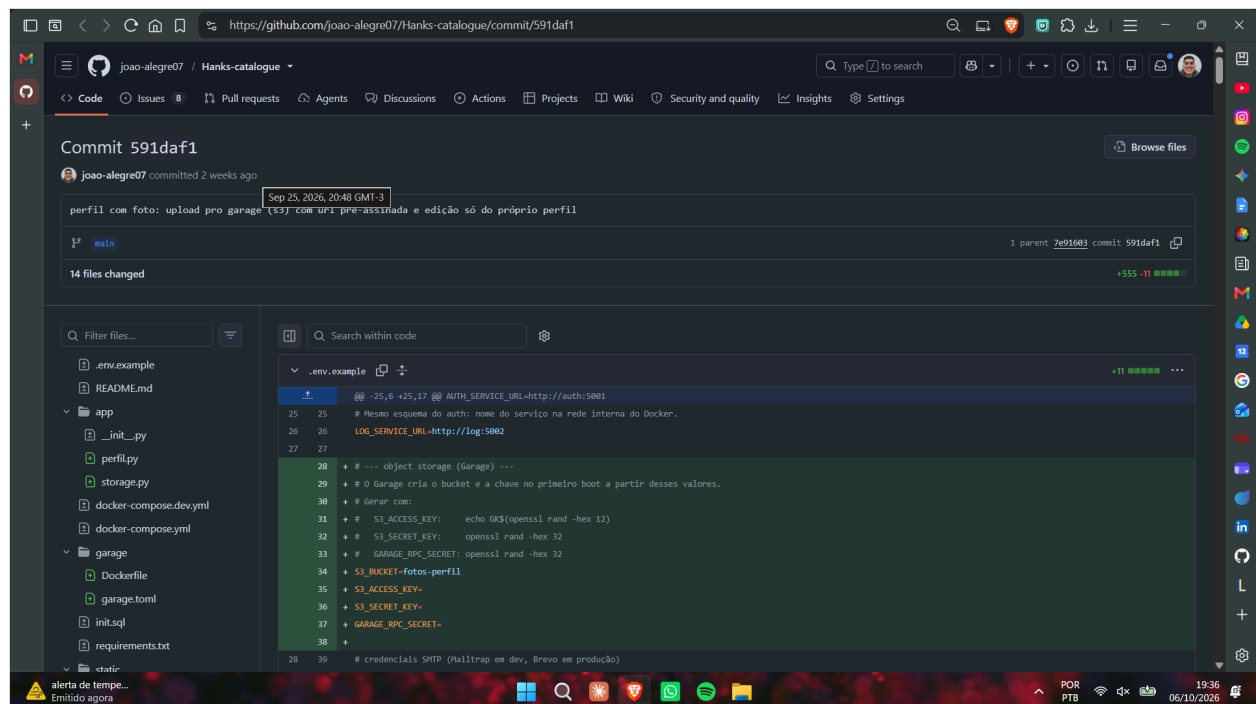


Figura 17: Atividade 6 — commit 591daf1, com a dica mostrando 25/09/2026 20:48 (GMT-3).

```
# Object storage das fotos de perfil (Garage, compatível com S3, no lugar  
# do MinIO). Também sem porta publicada: o catálogo envia as fotos e as  
# URLs assinadas chegam no navegador passando pela rota /fotos do catálogo.  
# O --default-bucket cria o bucket e a chave de acesso no primeiro boot a  
# partir das variáveis GARAGE_DEFAULT_*.  
garage:  
  build: ./garage  
  environment:  
    GARAGE_RPC_SECRET: ${GARAGE_RPC_SECRET}  
    GARAGE_DEFAULT_BUCKET: ${S3_BUCKET}  
    GARAGE_DEFAULT_ACCESS_KEY: ${S3_ACCESS_KEY}  
    GARAGE_DEFAULT_SECRET_KEY: ${S3_SECRET_KEY}  
  volumes:  
    - garage_data:/var/lib/garage  
  restart: unless-stopped
```

Figura 18: Atividade 6 — serviço garage no docker-compose.yml, sem porta publicada, com o bucket e as chaves vindos de variáveis de ambiente.

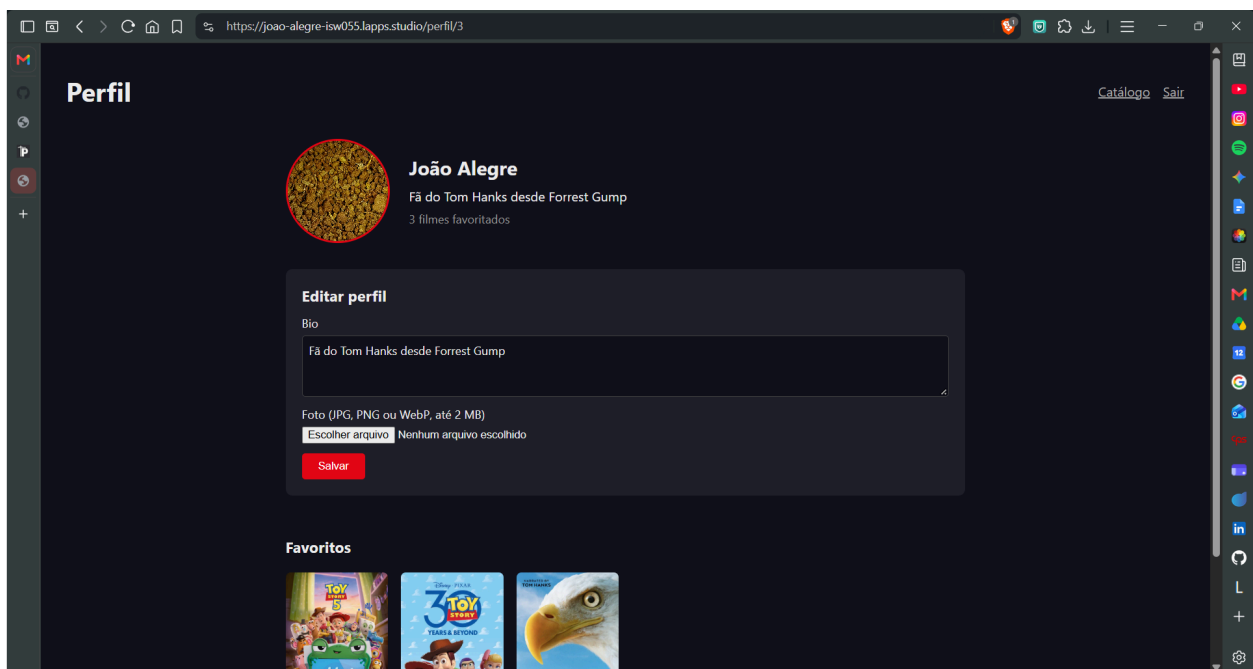


Figura 19: Atividade 6 — perfil em produção com a foto enviada, a bio e os filmes favoritos (30/09/2026).

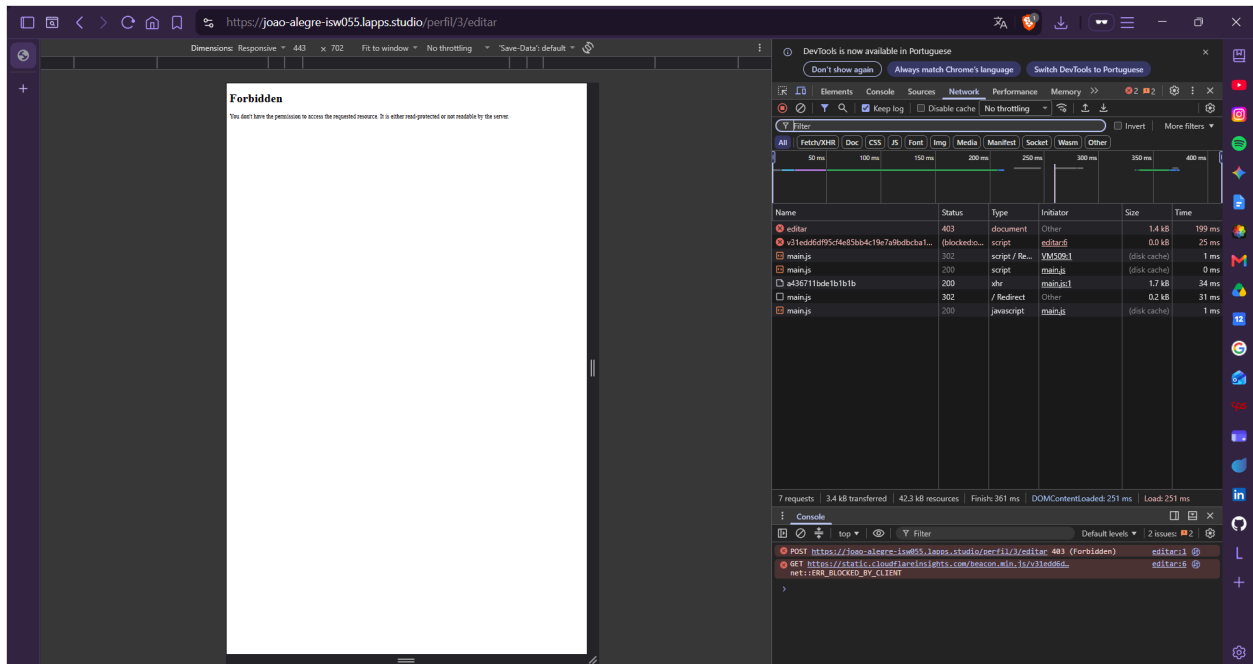


Figura 20: Atividade 6 — logado como outro usuário (id 17), troquei no DevTools o destino do formulário para /perfil/3/editar: o backend respondeu 403 e o perfil 3 não mudou.

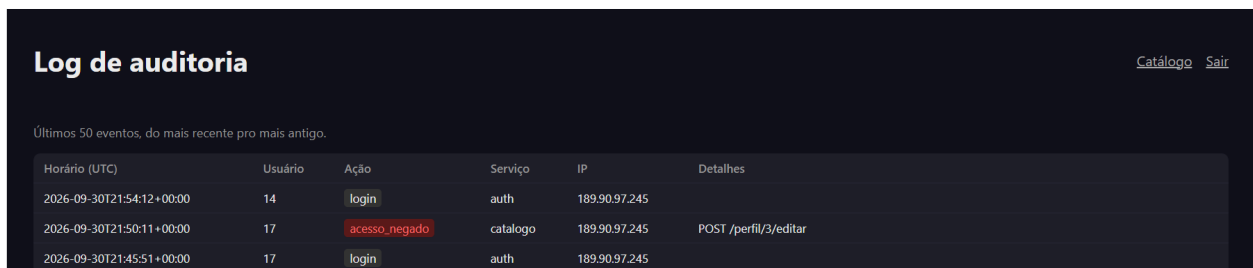


Figura 21: Atividade 6 — a tentativa registrada no log de auditoria como acesso_negado (POST /perfil/3/editar), em 30/09/2026 às 21:50 UTC.

Dificuldades e como foram resolvidas. Trocar o MinIO pelo Garage exigiu entender como ele monta um nó só e cria o bucket sozinho, para não depender de nenhum passo manual no servidor. O ponto mais difícil foi juntar bucket privado com um storage sem porta pública: a URL pré-assinada precisa chegar ao Garage com o mesmo caminho e a mesma query string, então a rota /fotos só repassa o pedido, sem decidir nada. Para provar o 403 na edição de outro perfil, como o formulário sempre aponta para o próprio perfil, alterei o destino dele no DevTools. Os prints e a issue foram feitos em 30/09, cinco dias depois do commit, ainda antes do prazo.

5 Atividades extras

5.1 Atividade E1 — Documentação Swagger/OpenAPI

Descrição	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.
Data planejada	sem prazo
Data realizada	01/10/2026 15:28 commit cd4c508 (git log); chegou ao GitHub às 15:28; issue #6 aberta às 16:19
Situação	entregue
Evidência	GitHub — commit, README e issue #6; Swagger UI público do catálogo
Link	https://github.com/joao-alegre07/Hanks-catalogue/commit/cd4c508 https://github.com/joao-alegre07/Hanks-catalogue/issues/6

O que foi feito. Documentei os três serviços em OpenAPI 3, com o Swagger UI em /apidocs gerado pelo flasgger. No catálogo, a especificação fica num arquivo separado (app/openapi.yml), porque várias rotas atendem GET (a página) e POST (o formulário) na mesma função; no auth-service e no log-service, ela fica em docstrings YAML em cada rota. Cada rota tem método, parâmetros, corpo esperado e as respostas possíveis com exemplo, inclusive os erros (400, 401, 403, 404, 409, 413 e 503, conforme a rota). O Swagger do catálogo é público, em joao-alegre-isw055.lapps.studio/apidocs os outros dois só abrem localmente, nas portas 5001 e 5002 do compose de desenvolvimento, porque esses serviços continuam sem porta pública em produção.

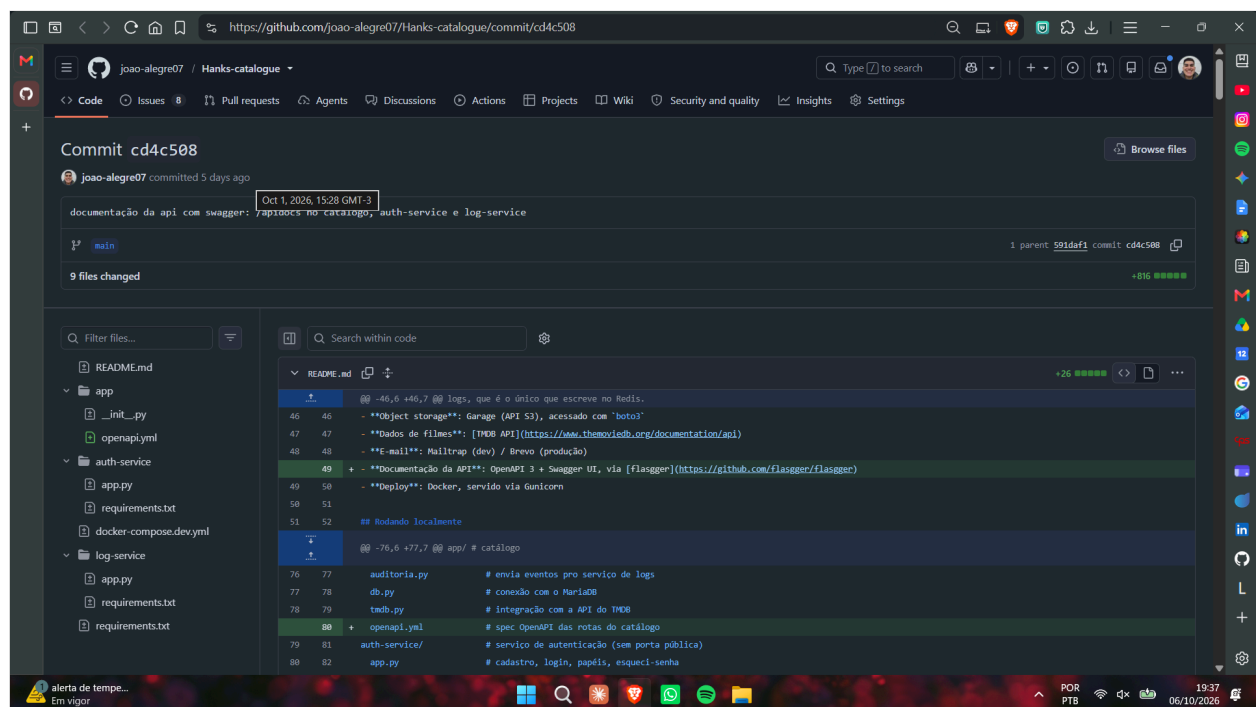


Figura 22: Extra E1 — commit cd4c508, com a dica mostrando 01/10/2026 15:28 (GMT-3).

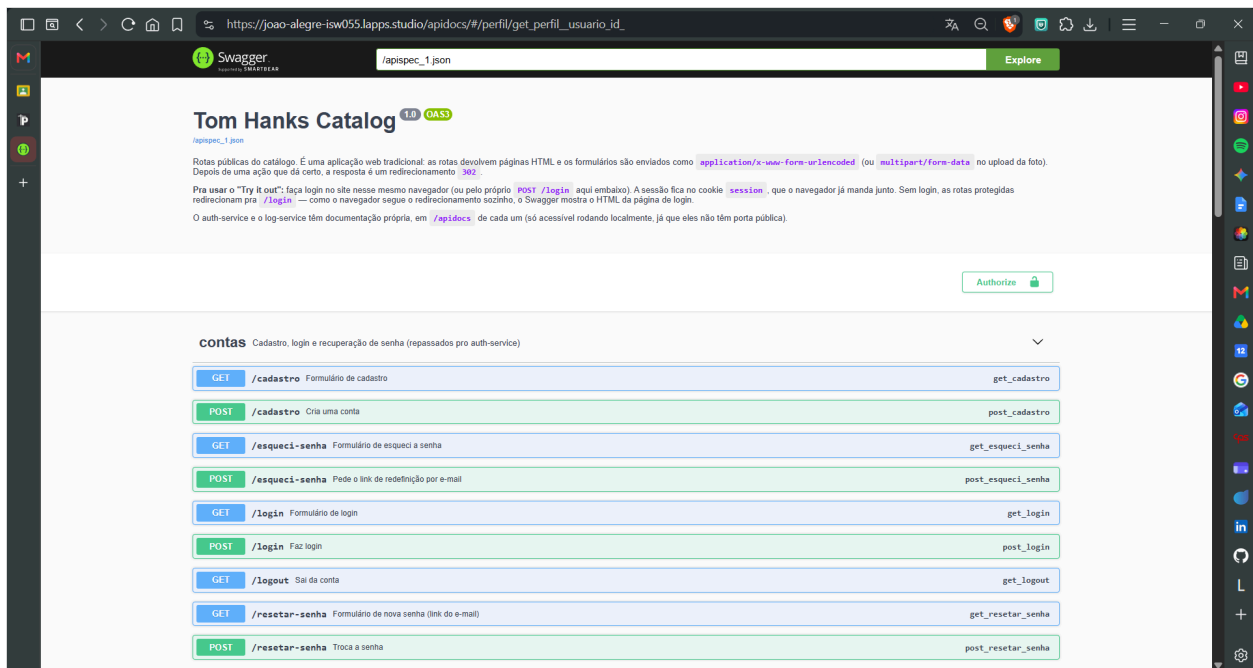


Figura 23: Extra E1 — Swagger UI do catálogo em produção, com as rotas agrupadas por assunto.

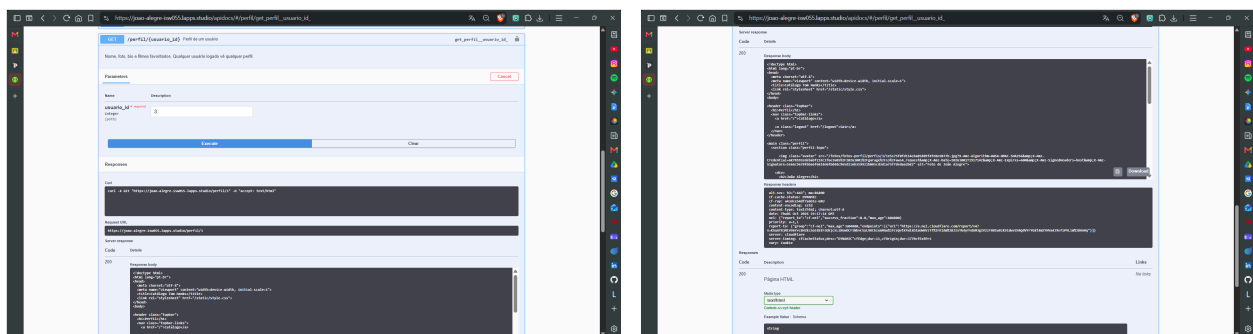


Figura 24: Extra E1 — “Try it out” em produção: GET /perfil/{usuario_id} executado com a sessão logada no navegador, com a resposta 200 e o HTML do perfil.

Dificuldades e como foram resolvidas. O catálogo não é uma API JSON: as rotas devolvem páginas HTML e recebem formulários. Em vez de forçar uma descrição que não corresponde ao sistema, descrevi as rotas como elas são (text/html, formulários e redirecionamentos 302) e expliquei no topo da página que o “Try it out” usa o cookie de sessão do navegador. No redeploy, o Portainer mostrou “Failure [object Object]” e o container do auth ficou com o estado *dead* por alguns minutos; ele voltou sozinho, e a partir daí passei a conferir o estado real dos containers antes de repetir um deploy.

5.2 Atividade E2 — CI/CD com GitHub Actions

Descrição	Pipeline que testa e faz deploy a cada push.
Data planejada	sem prazo
Data realizada	05/10/2026 18:49 commit 672ec5f (git log), que fechou o pipeline criado no 9201c83, às 16:30; chegou ao GitHub às 18:49; issue #7 aberta às 19:23
Situação	entregue
Evidência	GitHub — commit, workflow, README e issue #7; execução verde no Actions e containers com a tag do commit
Link	https://github.com/joao-alegre07/Hanks-catalogue/commit/672ec5f https://github.com/joao-alegre07/Hanks-catalogue/issues/7

O que foi feito. O workflow `.github/workflows/ci-cd.yml` roda a cada push na main, com três jobs em sequência. O primeiro roda o pytest dos três serviços (15 testes na época), sem depender de banco, Redis ou rede: as chamadas entre serviços são trocadas por respostas falsas e o Redis pelo fakeredis; se algum teste falha, nada é publicado. O segundo constrói as quatro imagens (catálogo, auth-service, log-service e Garage) e publica no GHCR com duas tags, `sha-<commit>` e `latest`. O terceiro faz o deploy: o compose de produção usa a imagem com a tag `${IMAGE_TAG}`, então o container sobe com a imagem exata de um commit e dá para saber o que está no ar olhando a tag.

Nenhum segredo fica no repositório. O token do Portainer está nos secrets do GitHub, as senhas do sistema continuam nas variáveis da stack no Portainer e o `.dockerignore` deixa o `.env` fora das imagens. A execução verde usada na entrega é a **37378493266**.

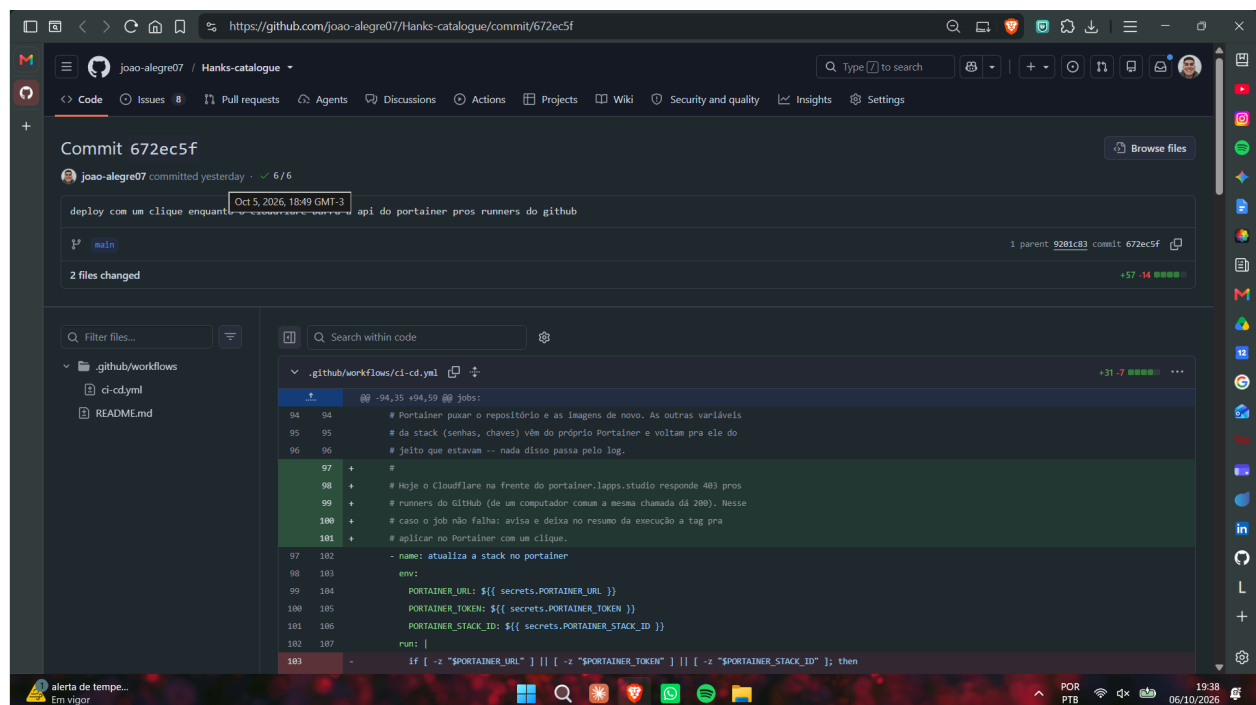


Figura 25: Extra E2 — commit 672ec5f, com a dica mostrando 05/10/2026 18:49 (GMT-3).

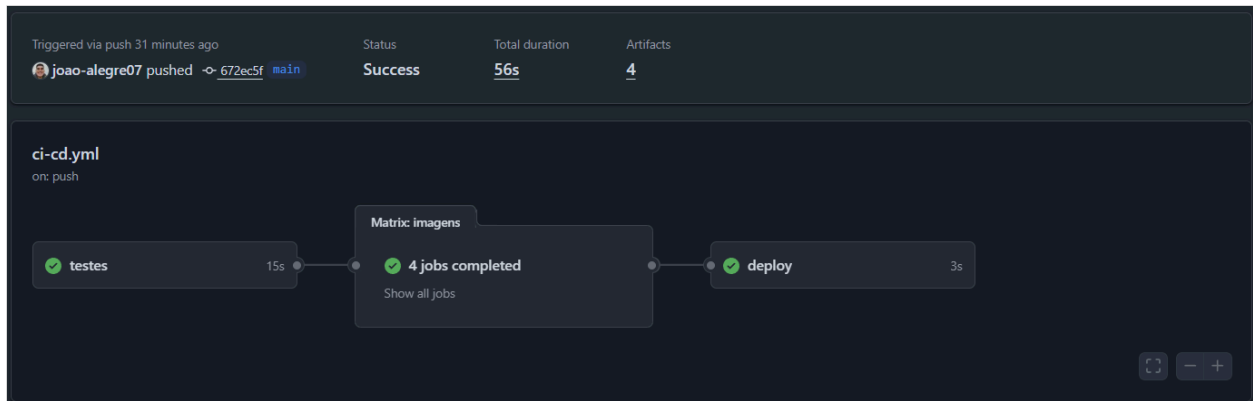


Figura 26: Extra E2 — execução do workflow para o commit 672ec5f: testes, as quatro imagens e o deploy concluídos com sucesso.

Containers								
Q Search... X [Start] [Stop] [Kill] [Restart] [Pause] [Resume] [Remove] [Menu]								
Name	State	Quick Actions	Stack	Image	Created	IP Address	Published Ports	Ownership
hanks-catalogue-app-1	running	[Stop] [Restart] [Kill] [Pause] [Resume]	hanks-catalogue	ghcr.io/joao-alegre07/hanks-catalogue-app:sha-672ec5f	2026-10-05 19:21:01	192.168.32.6	8212:5000	private
hanks-catalogue-auth-1	running	[Stop] [Restart] [Kill] [Pause] [Resume]	hanks-catalogue	ghcr.io/joao-alegre07/hanks-catalogue-auth:sha-672ec5f	2026-10-05 19:20:50	192.168.32.5	-	private
hanks-catalogue-garage-1	running	[Stop] [Restart] [Kill] [Pause] [Resume]	hanks-catalogue	ghcr.io/joao-alegre07/hanks-catalogue-garage:sha-672ec5f	2026-10-05 19:20:39	192.168.32.2	-	private
hanks-catalogue-log-1	running	[Stop] [Restart] [Kill] [Pause] [Resume]	hanks-catalogue	ghcr.io/joao-alegre07/hanks-catalogue-log:sha-672ec5f	2026-10-05 19:20:40	192.168.32.4	-	private
hanks-catalogue-redis-1	running	[Stop] [Restart] [Kill] [Pause] [Resume]	hanks-catalogue	redis:7-alpine	2026-10-05 19:20:39	192.168.32.3	-	private

Items per page: 10

Figura 27: Extra E2 — containers da stack no Portainer, recriados em 05/10/2026 às 19:20 com as imagens sha-672ec5f do GHCR.

Dificuldades e como foram resolvidas. No dia, o GitHub Actions estava com um incidente: as duas primeiras execuções do 9201c83 falharam com “job was not acquired by Runner”, sem nenhum teste rodar, e o pipeline só conseguiu executar cerca de duas horas depois. Quando rodou, testes e imagens passaram, mas o deploy recebeu 403 do Portainer. O mesmo token, testado do meu computador, recebeu 200: o bloqueio é do Cloudflare que fica na frente do Portainer da disciplina, que barra pedidos vindos dos servidores do GitHub. Como isso não depende de mim, no commit 672ec5f o job de deploy passou a avisar e deixar a tag do commit no resumo da execução, e o deploy virou um clique no Portainer (trocar o IMAGE_TAG e fazer *Pull and redeploy*). Também precisei deixar os pacotes do GHCR públicos, porque eles nasceram privados e o servidor não conseguia baixar as imagens.

5.3 Atividade E3 — Observabilidade — health checks e métricas

Descrição	Endpoints de saúde e métricas, com o container reagindo à queda do Redis.
Data planejada	sem prazo
Data realizada	05/10/2026 20:10 commit 1dafb25 (git log); chegou ao GitHub às 20:11; issue #8 aberta às 20:15; em produção às 20:20
Situação	entregue
Evidência	GitHub — commit, README com os prints e issue #8; /health e /metrics públicos do catálogo
Link	https://github.com/joao-alegre07/Hanks-catalogue/commit/1dafb25 https://github.com/joao-alegre07/Hanks-catalogue/issues/8

O que foi feito. Cada serviço ganhou duas rotas de saúde. O /health/live só diz que o processo responde; o /health testa de verdade as dependências, com timeout de 2 segundos, e responde 503 se alguma falhar: o catálogo faz SELECT 1 no MariaDB e HeadBucket no Garage, o auth-service faz SELECT 1 no MariaDB e o log-service faz PING no Redis. Os Dockerfiles têm HEALTHCHECK chamando o /health a cada 15 segundos, o Redis e o Garage têm o healthcheck no compose, e o compose só sobe o log-service depois que o Redis está saudável e o catálogo depois do Garage.

Os três serviços expõem /metrics no formato do Prometheus, com a contagem de requisições por rota e status e a latência. A label usada é a regra da rota (/perfil/<int:usuario_id>), e não o caminho real, para cada id não virar uma série nova. O compose de desenvolvimento sobe ainda um Prometheus e um Grafana com o painel já provisionado: requisições por minuto, taxa de erro 4xx/5xx e latência p95.

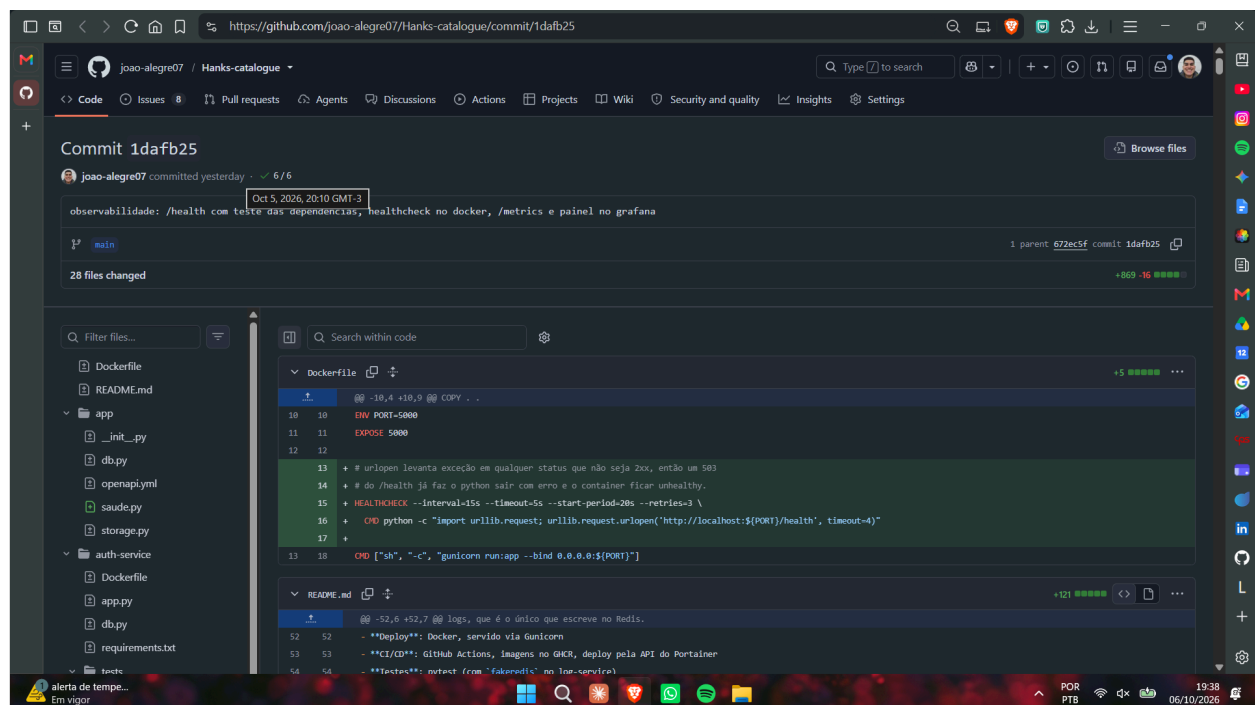


Figura 28: Extra E3 — commit 1dafb25, com a dica mostrando 05/10/2026 20:10 (GMT-3).

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\joaop> Get-Date; docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"

segunda-feira, 5 de outubro de 2026 19:54:01
NAMES                                IMAGE                                STATUS
atv2_catalogo_filmes-app-1           atv2_catalogo_filmes-app            Up 2 minutes (healthy)
atv2_catalogo_filmes-auth-1          atv2_catalogo_filmes-auth           Up 2 minutes (healthy)
atv2_catalogo_filmes-garage-1        atv2_catalogo_filmes-garage         Up 2 minutes (healthy)
atv2_catalogo_filmes-log-1           atv2_catalogo_filmes-log            Up 2 minutes (healthy)
atv2_catalogo_filmes-grafana-1       grafana/grafana:12.1.1              Up 3 hours
atv2_catalogo_filmes-prometheus-1    prom/prometheus:v3.5.0              Up 3 hours
atv2_catalogo_filmes-db-1            mariadb:11                          Up 3 hours (healthy)
atv2_catalogo_filmes-redis-1         redis:7-alpine                      Up 3 hours (healthy)

PS C:\Users\joaop> |

```

Figura 29: Extra E3 — docker ps com todos os serviços healthy, no ambiente local.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\joaop> docker stop atv2_catalogo_filmes-redis-1
PS C:\Users\joaop> Get-Date; docker ps -a --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"; curl.exe -s -w " HTTP %{http_code}" localhost:5002/health

segunda-feira, 5 de outubro de 2026 19:58:02
NAMES                                IMAGE                                STATUS
atv2_catalogo_filmes-app-1           atv2_catalogo_filmes-app            Up 6 minutes (healthy)
atv2_catalogo_filmes-auth-1          atv2_catalogo_filmes-auth           Up 6 minutes (healthy)
atv2_catalogo_filmes-garage-1        atv2_catalogo_filmes-garage         Up 6 minutes (healthy)
atv2_catalogo_filmes-log-1           atv2_catalogo_filmes-log            Up 6 minutes (unhealthy)
atv2_catalogo_filmes-grafana-1       grafana/grafana:12.1.1              Up 4 hours
atv2_catalogo_filmes-prometheus-1    prom/prometheus:v3.5.0              Up 4 hours
atv2_catalogo_filmes-db-1            mariadb:11                          Up 4 hours (healthy)
atv2_catalogo_filmes-redis-1         redis:7-alpine                      Exited (0) About a minute ago
{"dependencias":{"redis":"falhou: ConnectionError"},"status":"falhou"}
 HTTP 503

PS C:\Users\joaop> |

```

Figura 30: Extra E3 — Redis parado com docker stop: em menos de um minuto o log-service fica unhealthy e o /health dele responde 503 dizendo qual dependência falhou, enquanto o resto continua healthy.



Figura 31: Extra E3 — painel no Grafana durante o teste: entre 19:57 e 20:00, a queda do Redis aparece como taxa de erro de 100% e latência alta no log-service, e tudo volta ao normal quando o Redis sobe de novo.

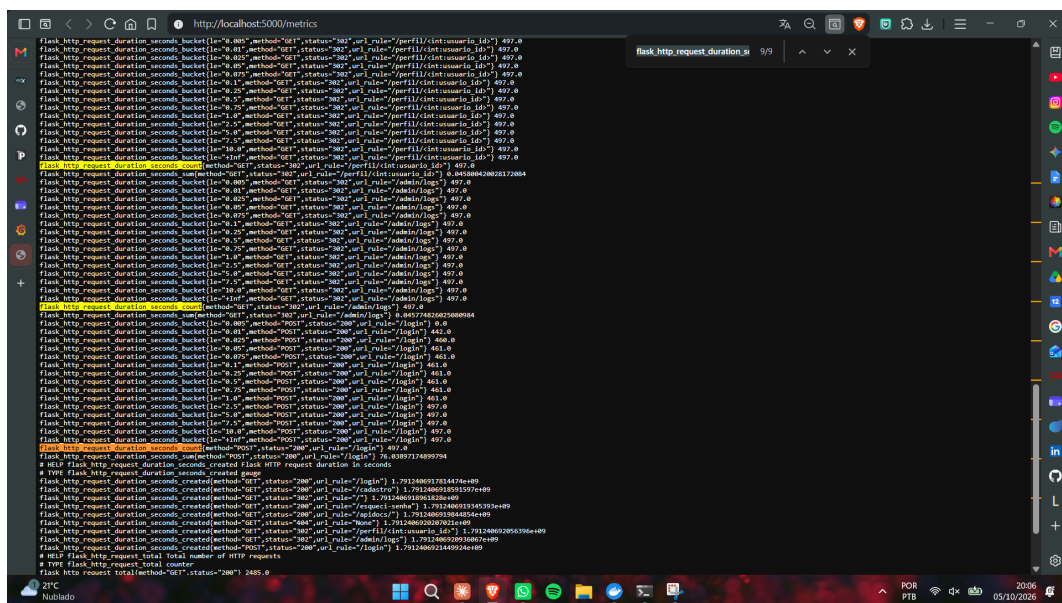


Figura 32: Extra E3 — /metrics do catálogo, com a contagem de requisições por rota e status.

Dificuldades e como foram resolvidas. A primeira decisão foi o que cada /health deveria testar. O catálogo não testa o auth-service nem o log-service, porque cada um tem o próprio; se testasse, uma queda do Redis deixaria dois containers unhealthy e o problema pareceria estar no lugar errado. No deploy em produção, o Portainer mostrou “Failure [object Object]”: com os healthchecks, a subida espera o Garage e o Redis ficarem saudáveis e passa do tempo limite de cerca de 100 segundos do Cloudflare. Conferi a lista de containers, todos recriados às 20:20 com a tag sha-1dafb25, e o /health de produção respondendo 200, e vi que o deploy tinha terminado no servidor. O Prometheus e o Grafana ficaram só no ambiente local, porque a stack da disciplina tem uma única porta pública.

6 Considerações finais

O bimestre mostrou na prática como uma aplicação simples vira um sistema distribuído, e que cada passo dessa mudança corresponde a um conceito de nuvem. O banco individual, sem permissão para criar outro schema, é o isolamento entre clientes visto do lado do cliente. Separar autenticação, auditoria e arquivos em serviços próprios, cada um dono dos seus dados, mostrou o que é desacoplar: o catálogo não sabe como a senha é guardada nem onde o evento de log é gravado, só conversa com cada serviço por HTTP. E deixar só o catálogo com porta pública, com todo o resto na rede interna do Docker, mostrou que a forma de expor os serviços já é uma decisão de segurança, tanto quanto o código.

O que mais exigiu de mim foi entender o porquê de cada decisão de arquitetura, e não só fazer funcionar. Em quase toda atividade havia mais de um caminho correto, com custos diferentes. Consultar o papel do usuário no auth-service a cada pedido faz a mudança de papel valer na hora, enquanto colocar o papel num token evita a chamada extra, mas a mudança demora a valer. Deixar o bucket público é mais simples e aproveita o cache, enquanto a URL pré-assinada só entrega a foto a quem está logado e expira. Se o serviço de log cai, é preciso escolher entre travar a ação do usuário ou deixá-la seguir sem o registro. Também precisei mudar a forma de pensar o sistema: quando o login sai de dentro da aplicação, toda chamada pode falhar pela rede, então timeout, resposta de erro e o comportamento com um serviço fora do ar passam a fazer parte do projeto. Foi isso que deu sentido às extras: o /health separa “o processo está vivo” de “o serviço consegue atender”, e o pipeline garante que o que está em produção é exatamente um commit que passou nos testes.

Outro aprendizado foi não confiar em nada que vem do navegador. O dono de um favorito vem da sessão definida pelo servidor, e não de um id enviado na requisição. O 403 de apagar o comentário de outra pessoa ou editar outro perfil é decidido no backend, e por isso testei chamando as rotas direto, pelo Postman e pelo DevTools, em vez de só conferir que o botão sumiu da tela. E o tipo da foto é conferido pelos primeiros bytes do arquivo, porque a extensão e o Content-Type podem ser forjados.

O que eu faria diferente é começar pelos testes automatizados. Os testes, o pipeline e o /health vieram com as atividades extras, que o professor publicou em 10/09, quando a atividade 4 já tinha sido entregue, e pelo cronograma da disciplina eles ficaram naturalmente para o fim do bimestre. Mesmo assim, olhando para trás, teria valido trazer essas práticas para o início do projeto. Até a extra de CI/CD, toda a validação era manual, pelo navegador e pelo Postman, e a cada atividade nova as regras das anteriores só eram conferidas de novo se eu lembrasse de testá-las. Os testes que entraram com o pipeline cobrem justamente esse tipo de regra (login, papel padrão no cadastro, token de senha de uso único, 403 ao editar outro perfil) e deveriam ter nascido junto com cada uma delas. O mesmo vale para o /health, que chegou quando o sistema já tinha cinco containers. Como próximos passos, quero acrescentar o tracing com OpenTelemetry, para acompanhar uma requisição passando pelo catálogo, pelo auth-service e pelo log-service, e levar esse mesmo cuidado com falhas para o Plano Premium do próximo bimestre, em que a confirmação do pagamento chega por webhook, de forma assíncrona.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Documento assinado digitalmente
 **JOAO PEDRO MORENO ALEGRE**
Data: 06/10/2026 20:46:27-0300
Verifique em <https://validar.it.gov.br>

João Pedro Moreno Alegre

Pompeia, 06/10/2026